



INDIA.RUNS

Build what next India runs on

Team Name : Abhinova

Team Leader Name : Abhinav Saxena

Problem Statement : Intelligent Candidate Discovery & Ranking

Solution Overview

A JD-adaptive hybrid ranker that reads careers, not keywords.

- Parses ANY job description into a target role-domain, experience band, location and excluded specialties, then ranks all 100K candidates and returns the top-100 with plain-language reasoning.

What makes it different from keyword / embedding matching:

- Reads the candidate's career prose (titles + history); treats the skills[] list as noise, so it is immune to the keyword-stuffer trap built into the dataset.
- Generalises to any role with no per-JD tuning: an AI JD surfaces ML engineers, a marketing JD surfaces marketers.
- Behavioural availability + platform-verified signals + a honeypot gate rank who is actually hireable, not just good on paper.
- CPU-only, no network or LLM calls at rank time, ~2 minutes for 100K, production-realistic.

JD Understanding & Candidate Evaluation

Requirements parsed automatically from the JD (JDSpec):

- Target role-domain, experience band (e.g. 5-9 yrs), location (Pune/Noida/India), salient keywords, and explicit exclusions from the 'do NOT want' section (CV/speech, consulting-only, title-chasers).

Signals that decide relevance, in priority order:

- Current title + full career history, the trustworthy role identity (dominant signal).
- Prose evidence of real work in role descriptions, not the self-reported skills list.
- Product-vs-services history, experience band, location, behavioural availability, verified ability (assessment scores, GitHub).

Beyond keywords: a 'Software Engineer' who shipped a recommender ranks like an ML engineer; a 'Marketing Manager' with a stuffed AI skill list does not.

Ranking Methodology

Retrieve -> score -> rank

- Parse JD into a JDSpec; build each candidate's domain vector from titles + prose; $\text{role_fit} = \cosine(\text{candidate vector}, \text{JD vector})$.
- Base score = interpretable weighted sum (=1): role 0.36 | semantic TF-IDF 0.18 | JD-keyword evidence 0.16 | experience 0.10 | company-type 0.10 | location 0.10.

Then multiplicative authenticity & availability gates:

- $x \text{ behavioural availability} \times \text{verified corroboration} \times \text{keyword-stuffing penalty} \times \text{excluded-specialty penalty} \times \text{job-hopping} \times \text{honeypot/consistency gate}$.

How signals combine:

- Weighted base = 'fit'; gates = 'is this real and reachable'. A great-on-paper but impossible or unreachable profile is pulled down, never up. Scores are non-increasing by rank; ties broken by candidate_id.

Explainability & Data Validation

Every ranked candidate gets a grounded reason + an auditable score breakdown.

- Reasoning is assembled only from facts in that candidate's own profile: title, years, JD-keywords found in their prose, location, and behavioural values.

Preventing hallucination:

- Template-free generation from those fields only, never cites a skill the profile lacks; tone matches the rank and surfaces honest concerns.

Suspicious / low-quality profiles:

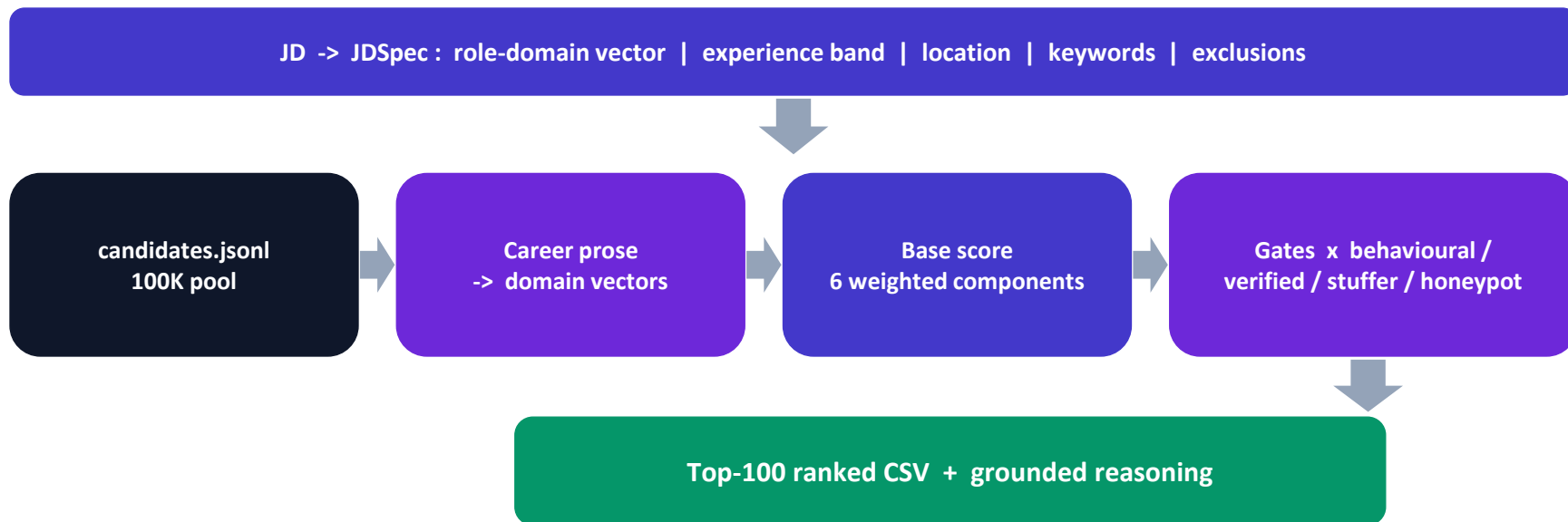
- An internal-consistency / honeypot gate catches impossible profiles (a role longer than the whole career, 'expert' skills with 0 months used, tenure beyond the role's own dates) and forces them out of the top-100.
- Result: 0 honeypots in our top-100 (disqualification threshold is 10). Keyword-stuffers are caught by the role-vs-skills mismatch penalty.

End-to-End Workflow

- 1 JD text -> parse to JDSpec (role-domain, experience band, location, keywords, exclusions).
- 2 Stream 100K candidates -> extract career prose -> build per-candidate domain vectors.
- 3 Fit TF-IDF index -> score every candidate (weighted base components x multiplicative gates).
- 4 Normalise & sort -> select top-100 -> generate grounded 1-2 sentence reasoning.
- 5 Write submission.csv (passes the official validate_submission.py unchanged).

The same rank_candidates() pipeline powers both the submission CSV and the interactive Streamlit sandbox demo.

System Architecture



Results & Performance

Ranking quality, top-100 on the released pool:

- Every pick is a genuine ML/AI/IR role | 100% India-based | 54/100 in the ideal 6-8 yr band.
- 0 honeypots in the top-100 (disqualification threshold is 10) | 0 off-domain CV profiles.
- Generality proven: the same code on a Marketing JD returns 61 Marketing Managers + 39 Content Writers, avg 8.1 yrs, Mumbai.

Runtime & compute, all challenge constraints met:

- ~2 min for 100K (<=5 min budget) | <=16 GB RAM | CPU only, no GPU | no network/LLM calls at rank time.
- Output passes the official validate_submission.py: 'Submission is valid.'

Technologies Used

- Python 3.11-3.14 - core implementation.
- scikit-learn - TF-IDF + cosine semantic match (fast, deterministic, CPU-only).
- NumPy / pandas - vectorised scoring and data handling.
- sentence-transformers (optional) - offline dense-embedding booster, blended if present.
- Streamlit + Altair - interactive sandbox with live weight tuning and explainability charts.
- reportlab - auto-generated approach report.

Why these: every choice is CPU-only and offline-capable to meet the no-network / 5-minute production constraint. No hosted LLM, so the system scales to 200K+ candidates in production.

Submission Assets

- GitHub repo: github.com/Abhi-tech-geek/redrob-ranker
- Ranked output: `submission.csv` - top-100, passes the official validator.
- Live sandbox: Streamlit demo (link in the repo README / portal).
- Reproduce: `python rank.py --candidates ./candidates.jsonl --out submission.csv`
- Approach deck + full methodology README included in the repo.



INDIA.RUNS

Build what next India runs on

THANK YOU